ELO320 - Estructuras de Datos y Algoritmos
Introducción a C
Estructura Base

Dr. Nicolás Gálvez R.
nicolas.galvez@usm.cl

Ing. Civil Telemática - Departamento de Electrónica

1er Semestre 2025

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

C Estructura general

- Requiere un método de inicio de ejecución: **main**

C

```c
#include<stdio.h>

int main() {
        printf("Hola Mundo\n");
        return 0;
}
```

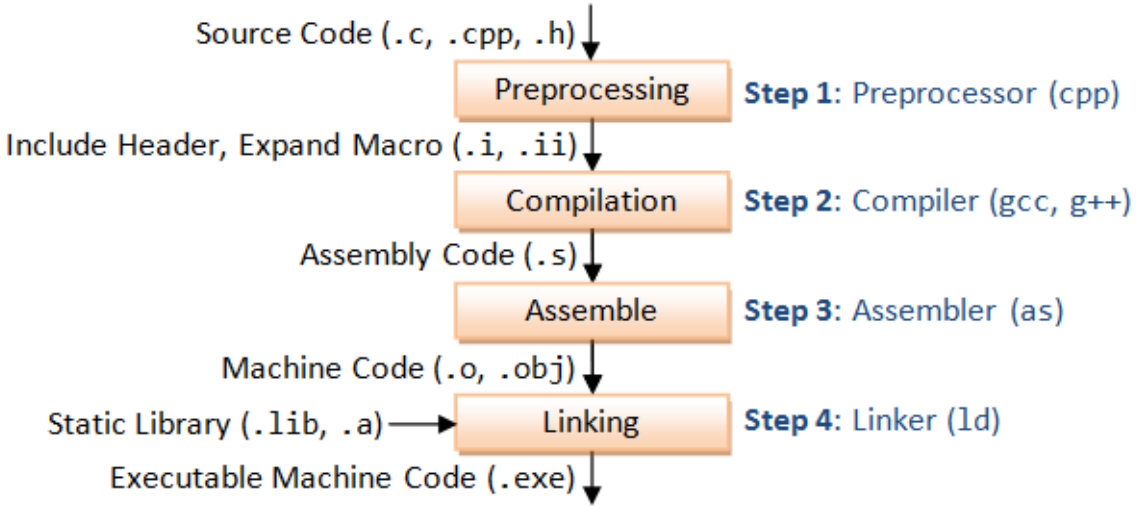
Python

```python
if __name__ == "__main__":
        print("Hola Mundo!")
```

Compilación: gcc

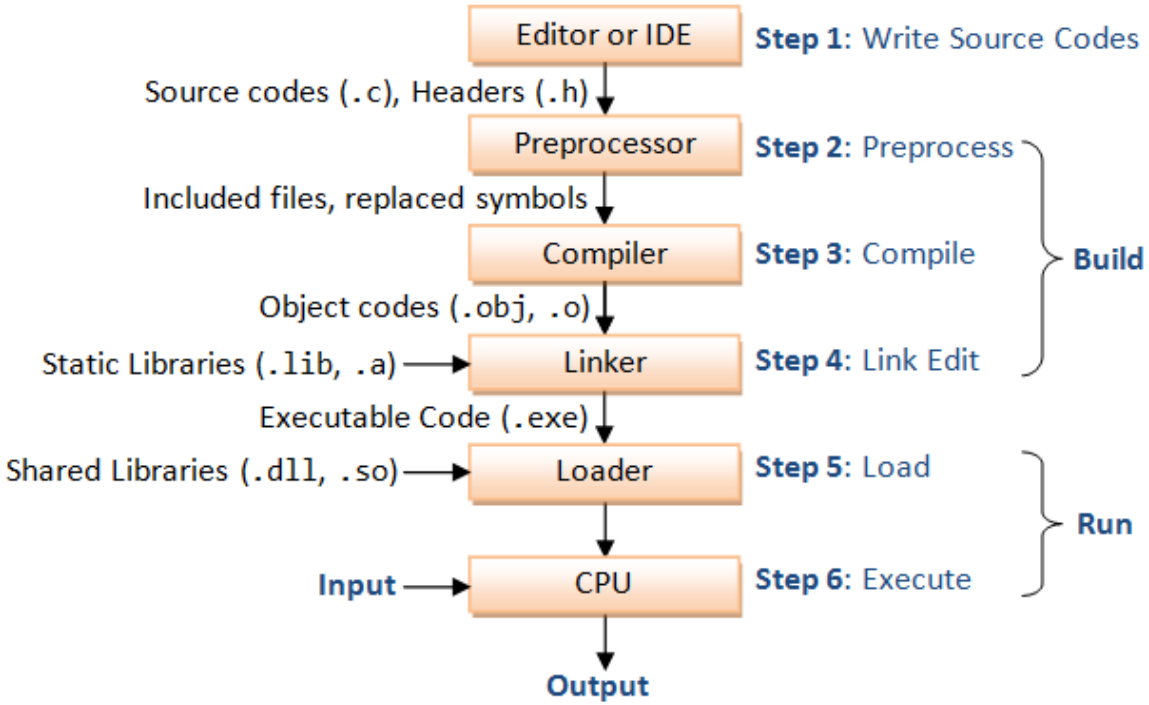
C es un lenguaje compilado, esto significa que:

- Su código es transformado, a través de un compilador, a lenguaje máquina.
- El resultado de la compilación es un binario ejecutable.
- Binarios ejecutables: .bin, .exe, .appinstall, etc.



Compilación: gcc

Quizás la imagen anterior ahora tenga mayor sentido:



Compilación: gcc

Compliación directa

```
[user@pc dir]$ gcc -o ejecutable codigo1.c codigo2.c -Wall
```

donde:

- **-o** (output): indica que lo siguiente es el nombre del ejecutable.
- **codigoX.c**: es el archivo de nuestro código (puede ser más de uno).
- **-Wall**: indica al compilador que de aviso de potenciales peligros (warnings).

Compilación: gcc

Compilación por pasos

```
[user@pc dir]$ gcc -c codigo1.c -Wall
[user@pc dir]$ gcc -c codigo2.c -Wall
[user@pc dir]$ gcc -o ejecutable codigo1.o codigo2.o
```

donde:

- **-c**: indica que un archivo.c será compilado independientemente a un archivo objeto .o.
- **-o** (output): indica que lo siguiente es el nombre del ejecutable.
- **-Wall**: indica al compilador que de aviso de potenciales peligros (warnings)
- En la última instrucción se enlazan todos los archivos compilados, de forma independiente, en un ejecutable.

Tipificación de Datos - Explícita VS Implícita

- Declaración exigida para detectar potenciales errores de tipo.
- **Tipificación explícita (C):**
  - Siempre detecta errores de tipo.
  - Restricciones sobre operaciones y valores de diferentes tipos.
  - No hay ejecución si tipos son erróneos o incompatibles.
  - Tipificación estática y explícita.
- **Tipificación implícita (Python):**
  - Conversiones que permiten relajar restricciones.
  - Flexibilidad y agilidad.
  - Genera errores no detectados.

Tipos de datos en C

- Define el tipo de dato de cada variable antes de usarlas (tipificación explícita):

C

```c
#include<stdio.h>

int main() {
        int a;
        float d;
        int b = 5, c = 6;
        printf("%d\n", b+c);
        return 0;
}
```

- En Python, los tipos de datos se infieren de la asignación (tipificación implícita).

Tipos de datos primitivos en C

Descripción	C	Python
Booleano	bool	bool
Byte	char	int
Caracter	char	str
Entero corto	short	int
Entero	int	int
Entero largo	long	long
Punto flotante	float	float
Pto. flotante largo	double	float
String	-	str
Complejos	-	complex

Tipos de datos primitivos en C

tipo	bytes	Rango
char	1	-127 to 127 or 0 to 255
unsigned char	1	0 to 255
signed char	1	-127 to 127
int	4	-2147483648 to 2147483647
unsigned int	4	0 to 4294967295
signed int	4	-2147483648 to 2147483647
short int	2	-32768 to 32767
unsigned short int	2	0 to 65,535
signed short int	2	-32768 to 32767
long int	4	-2,147,483,647 to 2,147,483,647
signed long int	4	same as long int
unsigned long int	4	0 to 4,294,967,295
float	4	+/- 3.4e +/- 38 (7 digits)
double	8	+/- 1.7e +/- 308 (15 digits)
long double	8	+/- 1.7e +/- 308 (15 digits)

Tipos de Datos

- **Sistema de tipificación en C:**
  - **Verificación de tipo:** Los operandos de una operación deben ser de *tipo compatible*.
  - **Tipo compatible:** es un tipo permitido, o que puede ser convertido en uno permitido.
  - **Conversión de tipos:** *Coerción* es conversión automática y *casting* es la conversión definida por el programador.

```
int pi = 3.14;              //coerción
double d = (double) pi;     //casting
```

  - **Error de tipo:** aplicación de un tipo inapropiado en una instrucción.

Especificadores

Las funciones de I/O de C utilizan especificadores para identificar el tipo de dato a leer o imprimir:

Tipo	Especificador
char	%c
int	%d %i
unsigned int	%u
float	%f %.3f
string	%s
double	%lf
long int	%ld
long long unsigned int	%Lu

Entrada Estándar: scanf

Función de entrada estándar, compuesta de una sección de especificadores y otra sección de variables, a la que se les asignará un valor obtenido desde pantalla.

scanf

```
int a;
char c;
char nom[10];
double prec;

scanf("%d", &a); //lee un entero
scanf("%c", &c); //lee un char
scanf("%s", nom); //lee un string

//lee un double, un string y un entero.
scanf("%lf %s %d", &prec, nom, &a);
```

Salida Estándar: printf

Función de salida estándar, compuesta de una sección de especificadores y otra de variables, cuyos valores se mostrarán por pantalla.

printf

```
printf("%d", a); //imprime un int
printf("%c", c); //imprime un char
printf("%s", nom); //imprime un string

//imprime un double de 3 decimales, un string y un entero.
printf("%.3lf %s %d", prec, nom, a);
```

Definir $\neq$ Asignar $\neq$ Inicializar

C

```c
#include<stdio.h>
int main(){
        int a; //Define a la variable a como tipo entero
        a = 3 + 2; //Evalúa 3+2 que es 5 y se asigna a 'a'.
        printf("%d\n", a);
        return 0;
}
```

Definir $\neq$ Asignar $\neq$ Inicializar

C soporta el uso de ambos conceptos (definición y asignación) en conjunto, llamado inicialización.

C

```
#include<stdio.h>
int main(){
        int a = 3 + 2;
        printf("%d\n", a); // Inicializa con resultado 3+2
        return 0;
}
```

Enumeración

C

```
enum color {
        rojo,
        amarillo,
        verde=20,
        azul};

int col = rojo;

if (col != azul){
        printf("No es azul\n");}
```

Python (clases)

```
class Colors(Enum):
        red = 1
        yellow = 2
        green = 3
```

Preview Próx. Clases: Arreglos y Strings

- **Arreglo:** Tipo estructurado consistente en conjunto homogéneo y ordenado de elementos.
- Se identifican por su posición relativa mediante un *índice* encerrado con paréntesis cuadrados (parte de índice 0).

```
//Arreglos estáticos (se liberan al terminar
// el ámbito donde fueron declaradas):
int foo[10]; //10 enteros creados en foo.
foo[3] = 7; //A cuarto elemento se asigna un 7.
```

- **String:** Arreglo de caracteres.

```
// Los string se delimitan con caracter '\0'
// (requiere espacio de 1 char):
char nombre[10] = "Juan"; //String de largo máximo 10.
nombre[1] = 'e'; //Qué hace?
printf("%s\n", nombre);
```

- Veremos más sobre arreglos la próxima clase.